Security Overview: Secure File Portal

---

1. Introduction

The **Secure File Portal** is designed to provide **confidentiality and integrity** for user files uploaded to a web-based system. The portal ensures that files are encrypted before storage and only decrypted when downloaded by authorized users.

This document outlines the **encryption methods, key management practices, and security considerations** implemented in the system.

---

2. Encryption Method

The portal uses **AES (Advanced Encryption Standard)** via the **Fernet module** from Python's cryptography library.

2.1 AES Encryption via Fernet

- **Algorithm:** AES-128 in CBC (Cipher Block Chaining) mode with HMAC authentication.
- **Key Length:** 32 bytes (base64 encoded) — meets the Fernet standard.
- **Purpose:** Ensures that all files are encrypted securely and cannot be read without the correct key.
- **Implementation:**

```
from cryptography.fernet import Fernet


# Load the AES key

fernet = Fernet(load_key())


# Encrypt a file

encrypted_data = fernet.encrypt(file.read())


# Decrypt a file

decrypted_data = fernet.decrypt(encrypted_data)
```

- **Encryption Flow:**
    1. User uploads a file through the web portal.
    2. File content is read as bytes.
    3. Fernet.encrypt() encrypts the bytes and adds authentication.
    4. The encrypted file is stored in the uploads/ folder.

5. On download, Fernet.decrypt() restores the file to its original form.

---

3. Key Handling

The AES encryption key is **critical to the security of the system**. Improper handling could compromise all encrypted data.

3.1 Key Generation

- A **32-byte random key** is generated on the first run using:

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()

with open("secret.key", "wb") as key_file:
    key_file.write(key)
```

- This ensures a **unique, high-entropy key** for each deployment.

3.2 Key Storage

- The key is stored in a **local file** named secret.key.
- Access to this file is **restricted**: the Flask app reads it in **binary mode** (rb) for encryption/decryption.
- **Best practice:** In production, keys should be stored in **environment variables** or a **secure vault** (e.g., AWS KMS, HashiCorp Vault).

3.3 Key Security Considerations

- Never commit secret.key to source control. Add it to .gitignore.
- Limit file permissions to only the app user.
- Regularly **rotate keys** if necessary; note that old files will require the old key to decrypt.

---

4. File Security

- Uploaded files are **never stored in plain text**.
- Even if an attacker gains access to the uploads/ folder, files are unreadable without the AES key.
- Decryption happens **only on user download**, keeping the system safe from unauthorized access.

---

5. Summary of Security Features

Feature	Implementation
File Encryption	AES via Fernet (AES-128 CBC + HMAC)
Key Generation	Auto-generated 32-byte base64 key
Key Storage	secret.key file; recommend vault/env variable
Confidentiality	Files encrypted at rest
Integrity	HMAC ensures encrypted file integrity
Access Control	Only Flask app reads key and encrypts/decrypts
Key Rotation / Best Practices	Periodic key rotation recommended for production

---

6. Recommendations for Production

1. **Move AES key to environment variable or secure vault** rather than local file.
2. **Enable HTTPS** for all user connections to protect data in transit.
3. **Implement authentication and access control** for uploading/downloading files.
4. **Regularly audit** the uploads/ folder and key access permissions.

✅ **Conclusion:**
The Secure File Portal uses **industry-standard AES encryption** and **careful key management** to ensure that all user files are stored and transmitted securely. By following best practices, this system protects data confidentiality and integrity while remaining simple to use.